

ARCHITECTURE DOCUMENT

Project: Visual Cognitive Coding Assistant with Multilingual Learning Support

1. Introduction

The **Visual Cognitive Coding Assistant with Multilingual Learning Support** is a web-based AI system that helps students improve their Data Structures and Algorithms (DSA) skills. Instead of giving direct answers, it analyzes a student's partial or incorrect code to understand their thinking process and provide guided feedback.

The system architecture is designed to support:

- Real-time code analysis
- AI-based mistake detection and pattern recognition
- Personalized learning profiles
- Visual explanations for optimization and time complexity
- Multilingual support
- Secure data storage and progress tracking

The overall design ensures smooth interaction between the user interface, AI engine, analysis modules, and database.

2. Development Methodology

The project follows the **Agile (Scrum) methodology**, allowing iterative development, testing, and improvement of AI components.

Sprint Flow of the Project

Sprint 1 – UI & Basic System Setup

- Design frontend interface (HTML, CSS, JavaScript)
- Create code upload interface
- Setup Flask backend framework
- Establish database connection

Sprint 2 – Core Backend & Preprocessing

- Implement code preprocessing (syntax check, formatting)
- Develop feature extraction module (loop depth, conditions, patterns)
- Store structured code features in database

Sprint 3 – ML Model Integration

- Integrate fine-tuned LLM for cognitive analysis
- Connect model with backend APIs
- Generate structured feedback (not full solution)

Sprint 4 – Error Pattern Memory & Personalization

- Implement mistake pattern comparison system
- Develop Code DNA profile generation
- Enable repeated error detection logic

Sprint 5 – Visualization & Multilingual Support

- Add optimization & time complexity visualizations (Plotly)
- Integrate multilingual explanation support
- Final system integration and testing

3. Architecture Style Selection

Considered Architectures:

Architecture	Reason Not Selected
Microservices	Too complex for an academic-level project and requires advanced deployment setup.
Event-Driven	The system does not require heavy asynchronous event handling.
Serverless	AI model integration and database tracking require continuous runtime and persistent backend control.

Selected Architecture: Modular Monolithic Architecture

Justification:

- Simple to develop, test, and deploy.
- Easily supports AI model integration and backend processing.
- Suitable for real-time code analysis and feedback generation.
- Simplifies debugging and system integration.
- Appropriate for an academic prototype with scalable internal modules.

Although the system follows a monolithic structure, it is internally divided into independent modules such as cognitive analysis, fingerprint extraction, error memory, visualization, and progress tracking.

4. System Overview

Working Flow:

- The student uploads partial or incorrect Python code through the user interface.
- The system preprocesses the code and extracts important structural features (loops, conditions, patterns).
- The AI model performs cognitive analysis to identify logical mistakes and inefficiencies.
- The Error Pattern Memory checks for repeated or similar past mistakes.
- The system generates guided feedback along with visual explanations of optimization and time complexity.
- The feedback and performance data are displayed to the student and stored for progress tracking.

5. Architectural Layers

5.1 Presentation Layer

Responsible for user interaction.

Components:

- Web Interface (HTML, CSS, JavaScript)
- Interactive Visualization (Plotly Charts)

Functions:

- Accept student's Python code submission
- Allow language selection for explanations
- Display guided feedback and mistake analysis
- Show optimization and time complexity visualizations
- Present student progress dashboard

5.2 Application Layer

Responsible for processing, intelligence, and adaptive feedback logic.

Components:

a. Flask API

Handles communication between frontend and backend.

b. Code Preprocessing & Feature Extraction

Extracts structural features and builds Code Fingerprint.

c. AI Cognitive Analysis Model

Analyzes logical mistakes and generates structured feedback.

d. Error Pattern Memory System

Detects repeated mistakes by comparing past submissions.

e. Learning Agent (Adaptive Feedback Module)

- Monitors student response patterns
- Tracks which feedback improves performance
- Adjusts explanation depth and style over time
- Improves personalization across submissions

f. Visualization & Feedback Engine

Generates optimization graphs and multilingual explanations.

Responsible for data storage and tracking.

Components:

- MySQL Database

Stores:

- Student code submissions
- Extracted feature data
- Error history
- Feedback results
- Student progress metrics
-

6. Module Description

Module	Description
UI	Collects student’s Python code input and displays personalized feedback, visualizations, and progress reports.
Flask Backend	Handles API requests and connects frontend with AI modules, fingerprint engine, and database.
Code Preprocessing & Fingerprint Engine	Performs syntax checking, extracts structural features (loops, conditions, patterns), and generates the student’s Code DNA profile.
AI Cognitive Analysis Model	Analyzes logical flow, detects conceptual mistakes, and generates structured guided feedback.
Error Pattern Memory System	Compares new submissions with stored mistake patterns to detect repeated or common logical errors.
Learning Agent (Adaptive Module)	Tracks student progress and adjusts feedback style based on past performance and misunderstanding patterns.
Visualization & Feedback Engine	Generates optimization graphs, time complexity visualizations, and multilingual explanations.
Database	Stores student submissions, extracted features, mistake history, feedback results, and progress metrics.

7. Data Flow

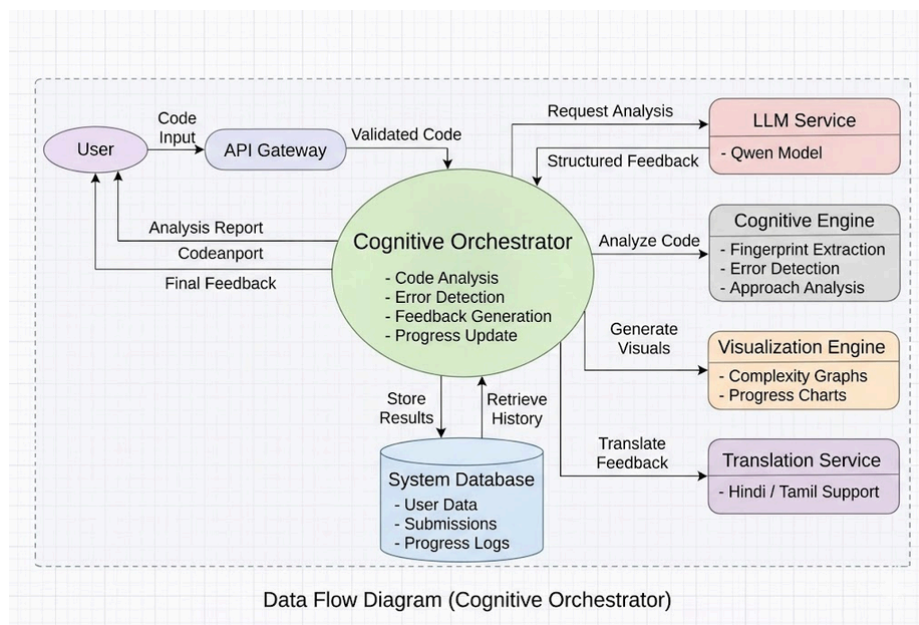


Fig 1: Data Flow Diagram

The Data Flow Diagram illustrates how data moves within the Visual Cognitive Coding Assistant. It begins when a student submits partial or incorrect Python code. The system preprocesses and analyzes the code to detect logical errors, patterns, and complexity issues. The Learning Agent updates the student's profile based on detected mistakes and generates personalized feedback with visual and multilingual explanations. Finally, the results are displayed to the student and stored for progress tracking.

8. UML Diagrams

8.1 Use Case Diagram

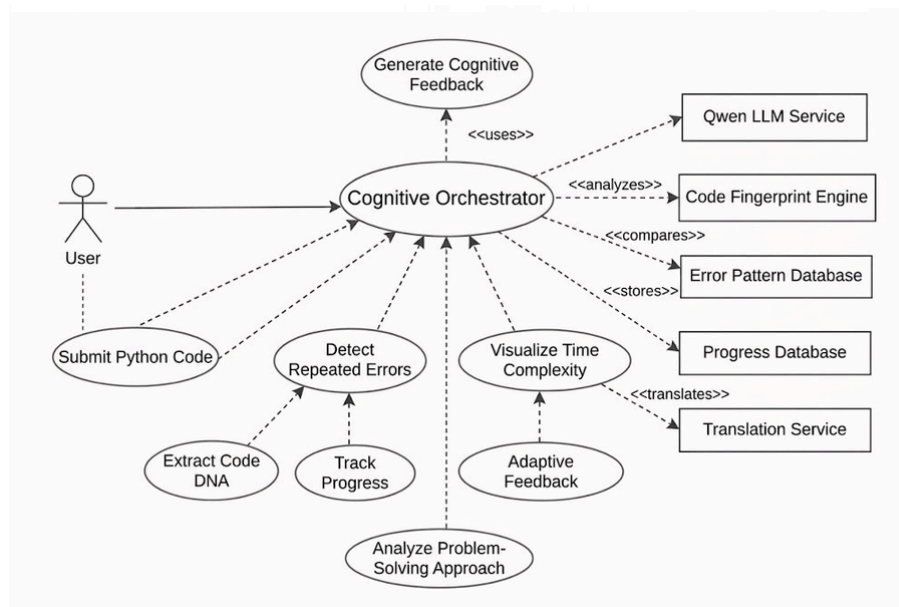


Fig 2: Use Case Diagram

The Use Case Diagram shows how the **Student** interacts with the system to upload code, receive feedback, view visual explanations, select language, and track progress. The **Admin** manages datasets and monitors system performance.

The Component Diagram presents the system modules: Frontend UI, Flask Backend, Cognitive Analysis Engine, Learning Agent, Visualization Module, and Database. It explains how these components work together

to analyze code and display results.

8.4 Sequence Diagram

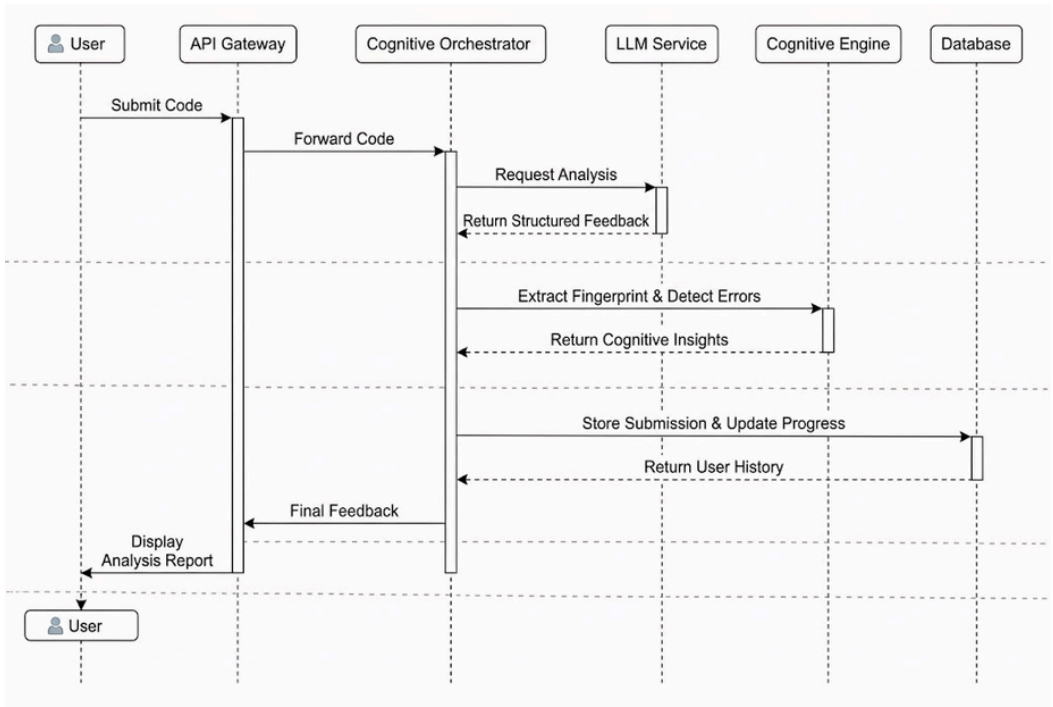


Fig 5: Sequence Diagram

The Sequence Diagram shows the step-by-step flow:
Student uploads code → Backend processes it → Cognitive analysis is performed → Learning Agent updates profile → Feedback is generated → Results stored and displayed.

8.5 Deployment Diagram

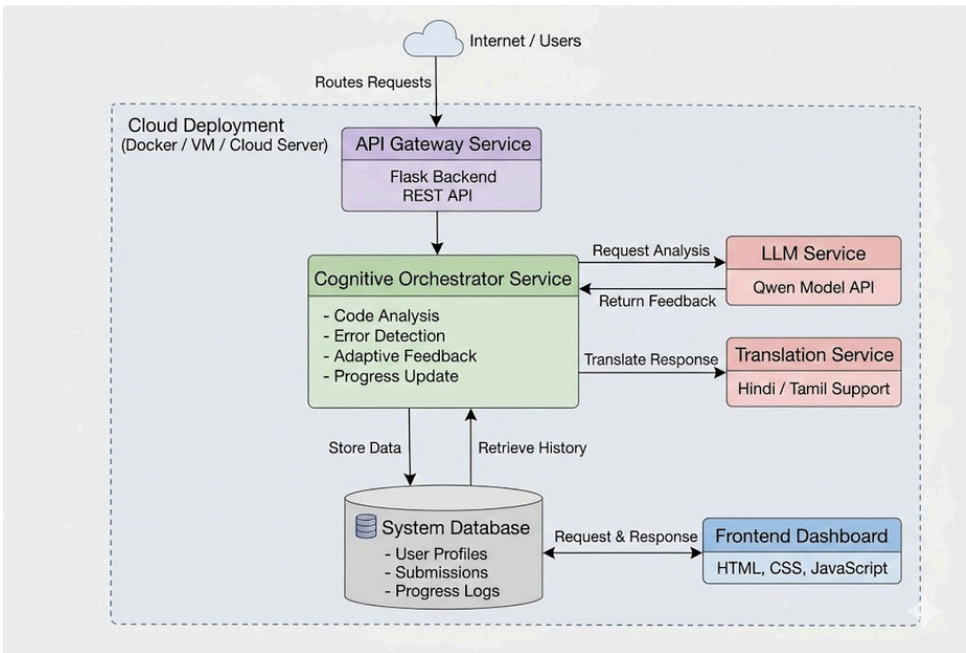


Fig 6: Deployment Diagram

The Deployment Diagram shows the Client Browser connected to a Flask Server that hosts AI modules and connects to a database for storing submissions and progress data.

9. Architectural Justification

The Modular Monolithic architecture:

- Supports AI integration
- Enables real-time analysis
- Easy to develop and deploy
- Suitable for academic prototype

10. Scalability Consideration

Future upgrades include:

- Cloud deployment
- Microservices transition
- Separate AI inference server
- Advanced adaptive learning

11. Conclusion

The architecture supports cognitive code analysis, personalized learning, multilingual feedback, and progress tracking while remaining simple and scalable.